



Faculty of Engineering and Technology
Department of Electrical and Computer Engineering
ENCS3390-Operating System Concepts
First Project Report

Naïve, Multiprocessing, multithreading approaches comparison

Prepared by: Lara Nidal Daifallah

Student ID: 1230239

Instructor: Dr. Mohammad Khalil

Section: 2

Date: 21/11/2025

Abstract

This project discusses three approaches to process a large sales dataset on Linux. It considers a sequential version, a multiprocessing version using `fork()`, and a multithreading version using `pthread`s. The programs read CSV files containing millions of sales records and calculate total revenue and profit based on user-selected filters such as region, country, item type, or sales channel. The report explains how each program loads and organizes data in memory, divides the workload across threads or processes, and manages synchronization to avoid conflicts, and time executed in each approach and case. It also examines the assumptions, design choices, and limitations of each approach. Additionally, Amdahl's Law is applied to estimate the theoretical speedup and compare it with observed performance, highlighting the effects of parallelism and system constraints.

Table of Contents

Abstract	II
List of Tables	IV
List of figures.....	IV
The environment of the work	5
1. Computer Specifications:	5
2. Virtual Machine Specifications (WSL / Linux Environment):	5
3. Programming Language and IDE Tool:	5
Theory	8
1. Naïve approach	8
2. Multiprocessing approach.....	8
3. Multithreading approach	8
Proceedure & Codes Implementaion.....	10
1. Naïve Approach	10
2. Multi-processing Approach.....	13
3. Multi-threading Approach:.....	16
Results	20
1. Naïve Appeoach	20
2. Multi-threading approach.....	21
3. Multi-processing Approach.....	23
Results Discussion:	26
Performance Measuring	27
Amdahl's Law Calculations (Theoretical Speedup)	28
Conclusion	31
References.....	32

List of Tables

<i>Table 1: Numerical Results</i>	27
<i>Table 2: Performance Table</i>	27
<i>Table 3: Actual vs. Theoretical Speedup</i>	30

List of figures

<i>Figure 1: Pc environment</i>	6
<i>Figure 2: Windows Specifications</i>	6
<i>Figure 3: Number of cores and threads in my pc</i>	6
<i>Figure 4: WSL environment</i>	7
<i>Figure 5: RAM and OS type for WSL</i>	7
<i>Figure 6: gcc --version run</i>	7
<i>Figure 7: Naive Results</i>	20
<i>Figure 8: 2 Threads</i>	21
<i>Figure 9: 4 Threads</i>	21
<i>Figure 10: 8 Threads</i>	22
<i>Figure 11: 16 Threads</i>	22
<i>Figure 12: 20 Threads</i>	22
<i>Figure 13: 2 Processes</i>	23
<i>Figure 14: 4 Processes</i>	24
<i>Figure 15: 8 Processes</i>	24
<i>Figure 16: 16 Processes</i>	24
<i>Figure 17: 20 Processes</i>	25
<i>Figure 18: Naive last attempt</i>	26

The environment of the work

1. Computer Specifications:

- **Processor:** AMD Ryzen 5 5600H with Radeon Graphics, 6 cores / 12 threads, 3.30 GHz base frequency
- **Memory:** 16 GB DDR4 RAM
- **Operating System:** Windows 11, 64-bit

Those Specifications are shown in figure1, figure2, figure3

I used a Virtual Machine:

2. Virtual Machine Specifications (WSL / Linux Environment):

- **Processor:** AMD Ryzen 5 5600H with Radeon Graphics
- **Architecture:** x86_64 (64-bit)
- **Cores:** 6 physical cores
- **Threads:** 12 (2 threads per core)
- **Operating System:** Ubuntu 24.04.3 LTS (GNU/Linux 6.6.87.2-microsoft-standard-WSL2 x86_64)
- **Total RAM:** 7.4 GiB

Those Specifications are shown in figure4, figure5

3. Programming Language and IDE Tool:

- **Language:** C
- **Compiler:** GCC 8.1.0 (MinGW-w64)
- **Development Tool / IDE:** Terminal-based text editor in Linux (Nano or Vim)

Those Specifications are shown in figure6

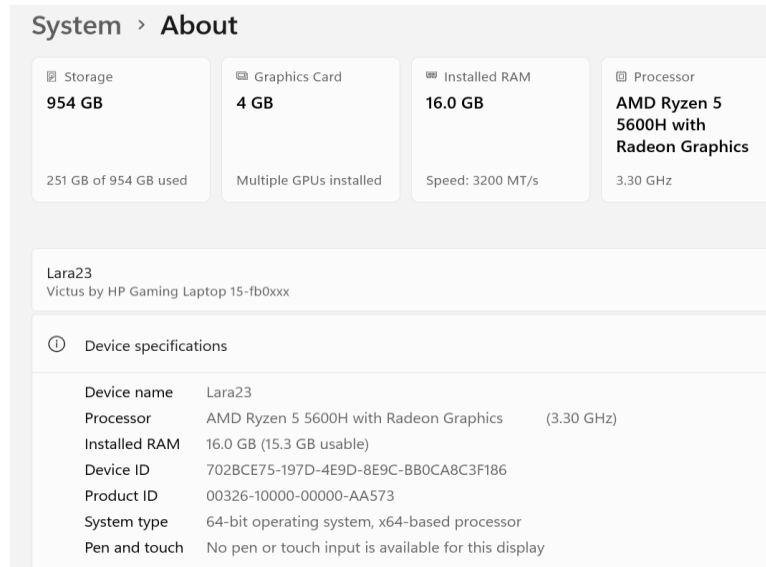


Figure 1: Pc environment

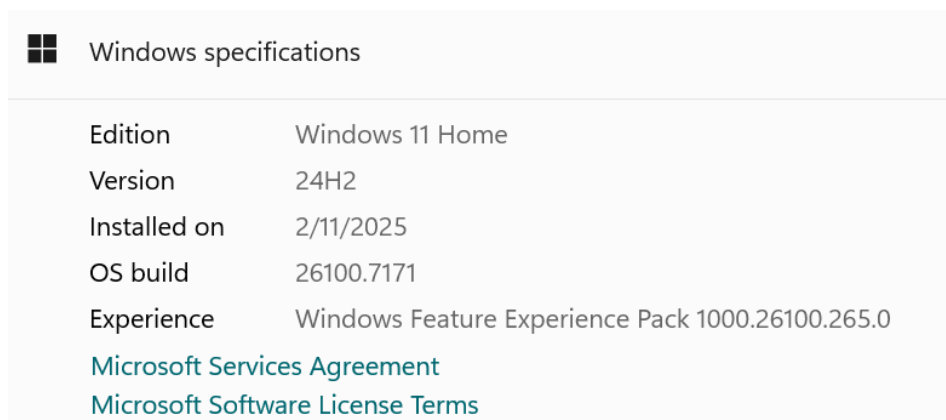


Figure 2: Windows Specifications

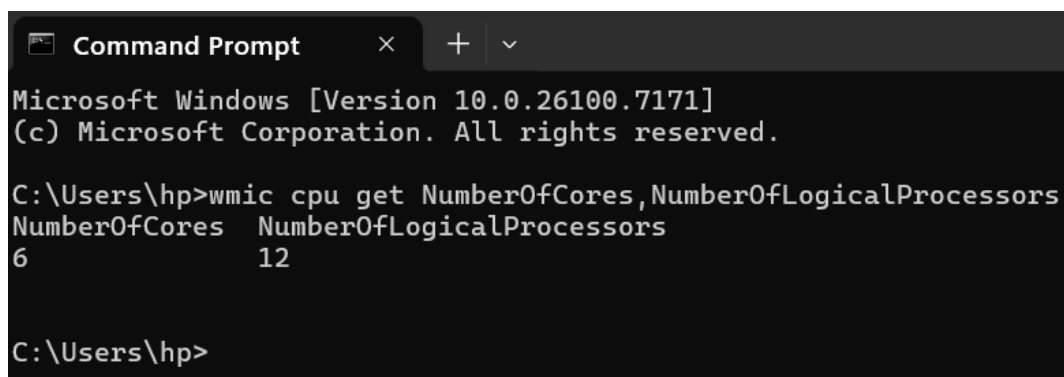


Figure 3: Number of cores and threads in my pc

```

lara@Lara23:/mnt/c/Users/hp$ lscpu
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Address sizes:          48 bits physical, 48 bits virtual
Byte Order:             Little Endian
CPU(s):                 12
On-line CPU(s) list:    0-11
Vendor ID:              AuthenticAMD
Model name:             AMD Ryzen 5 5600H with Radeon Graphics
CPU family:             25
Model:                  80
Thread(s) per core:     2
Core(s) per socket:     6
Socket(s):              1
Stepping:               0
BogoMIPS:               6587.44

```

Figure 4: WSL environment

```

lara@Lara23:/mnt/c/Users/hp$ uname -a
Linux Lara23 6.6.87.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun  5 18:30:46
UTC 2025 x86_64 x86_64 x86_64 GNU/Linux
lara@Lara23:/mnt/c/Users/hp$ free -h
              total        used        free      shared  buff/cache   available
Mem:           7.4Gi        523Mi        6.8Gi         3.8Mi         322Mi        6.9Gi
Swap:          2.0Gi          0B          2.0Gi

```

Figure 5: RAM and OS type for WSL

```

lara@Lara23:~/os_project$ exit
logout
PS C:\Users\hp> gcc --version
gcc.exe (x86_64-posix-seh-rev0, Built by MinGW-W64 project) 8.1.0
Copyright (C) 2018 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

```

Figure 6: gcc --version run

Theory

1. Naïve approach

The naïve approach uses a single process to carry out the entire task in a sequential manner. One execution flow handles all of the tasks, including reading the dataset, filtering records, and calculating totals. Although this approach guarantees deterministic behavior and is easy to implement, its performance is limited, particularly when dealing with large datasets. The program cannot benefit from contemporary multi-core processors because the CPU only manages one instruction stream. Because of this, when compared to multiprocessing or multithreading, the naïve approach usually shows the longest execution time.

2. Multiprocessing approach

In the parallel programming approach - multiprocessing, a program (parent) uses `fork()` to generate several independent child processes, each of which runs concurrently. Strong fault isolation and total prevention of race conditions are ensured by giving each process its own private memory space, which includes its own stack, heap, and global variables. Multiprocessing can operate much more quickly than the naïve (fully serial) approach because the workload is distributed among multiple processes, resulting in a shorter overall serial portion of the program, and so less time. Nevertheless, there is a cost associated with this speed increase: processes use more memory and switch context more slowly than threads. Because of parallel execution, multiprocessing is therefore faster than the naïve approach, but it is heavier and usually slower than multithreading [1] [2].

3. Multithreading approach

The multithreading approach allows multiple threads to function within the same process using libraries like Pthread (used in this project). Unlike multiprocessing, communication is extremely lightweight and fast because all threads share the same memory space, including global variables, the heap, and data structures. Because threads avoid the high memory and context-switching costs of processes, multithreading is typically faster than

both naïve and multiprocessing methods. However, synchronization are required to ensure accuracy strategies (I used mutex) because shared memory increases the risk of race conditions. Multithreading typically achieves high performance by dividing work among threads, reducing serial execution time, and enabling efficient parallelism at a lower overhead than multiprocessing [1] [3].

Procedure & Codes Implementaion

1. Naïve Approach

The code implementation is as follows:

```
lara@Lara23: ~/os_project
GNU nano 7.2
naive.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#define MAXLINE 200
#define MAXINPUT 40
#define NUMFILES 20
S_
//a struct to store a single sales record
typedef struct {
    char region[MAXINPUT];
    char country[MAXINPUT];
    char itemType[MAXINPUT];
    char salesChannel[MAXINPUT];
    int unitsSold;
    double unitPrice;
    double unitCost;
} SalesRecord;

//function prototypes
void showMenu();
void getUserInput(char *choice, char *filterValue);
void processFiles(const char *choice, const char *filterValue, int *totalOrders, double *totalRevenue, double *totalProfit);
void processFile(const char *filename, const char *choice, const char *filterValue, int *totalOrders, double *totalRevenue, double *totalProfit);
```

The first lines of my code include standard libraries, such as for input and output, for memory and utility functions, for handling strings, and for working with time, are included in the first lines of my code. To make managing my code easier, I then defined a few constants: MAXFILES is the total number of files in the project, MAXINPUT is the maximum length of text fields like region or country, and MAXLINE is the maximum length of a line from a file. I created a SalesRecord struct to hold all the data for a single record. In my program, this made it simple to arrange the data and distribute it. I also added Prototypes of functions.

```
lara@Lara23: ~/os_project
GNU nano 7.2

int main() {
    char choice;
    char filterValue[MAXINPUT];
    int totalOrders = 0;
    double totalRevenue = 0.0, totalProfit = 0.0;
    clock_t startTime, endTime;

    //show menu and ask user to choose a filter
    showMenu();
    getUserInput(&choice, filterValue);

    //start timing calculation
    startTime = clock();

    //process all CSV files
    processFiles(&choice, filterValue, &totalOrders, &totalRevenue, &totalProfit);

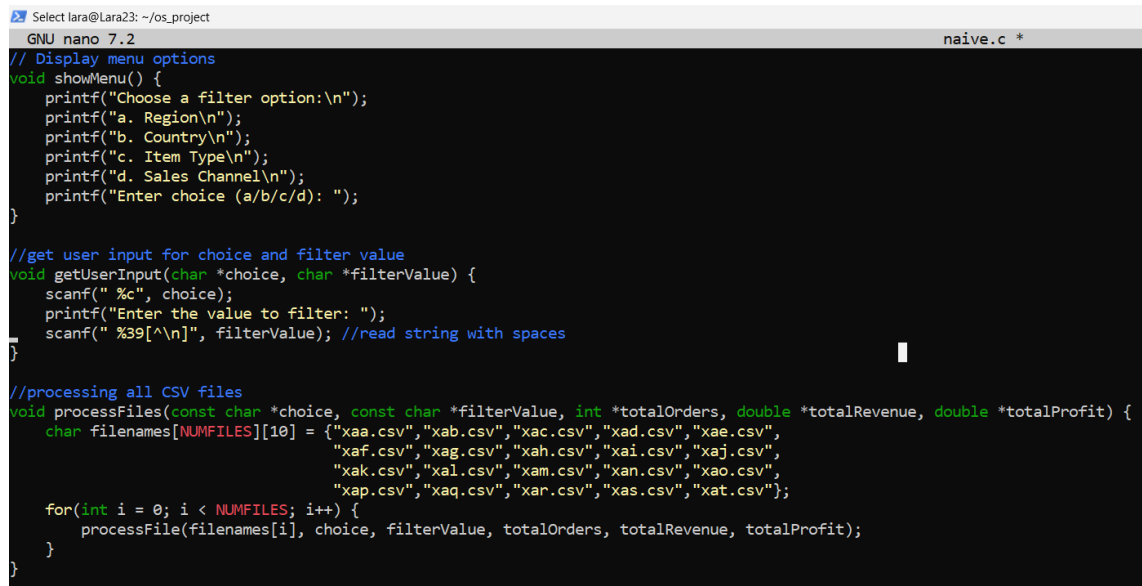
    //end timing
    endTime = clock();

    // show results
    printf("\nResults for \"%s\":\n", filterValue);
    printf("Total Orders: %d\n", totalOrders);
    printf("Total Revenue: %.2f\n", totalRevenue);
    printf("Total Profit: %.2f\n", totalProfit);

    double elapsedTime = ((double)(endTime - startTime)) / CLOCKS_PER_SEC;
    printf("Execution Time: %.3f seconds\n", elapsedTime);

    return 0;
}
```

This photo shows the implementation of the main function for naïve approach, I declared variables needed. I also added 2 variables to measure the execution time. First, I displayed the menu and got input from the user, then I started the timer and processed all the sales files to calculate the totals. After processing, I stopped the timer and displayed the results along with the execution time.



```
GNU nano 7.2                                     naive.c *
// Display menu options
void showMenu() {
    printf("Choose a filter option:\n");
    printf("a. Region\n");
    printf("b. Country\n");
    printf("c. Item Type\n");
    printf("d. Sales Channel\n");
    printf("Enter choice (a/b/c/d): ");
}

//get user input for choice and filter value
void getUserInput(char *choice, char *filterValue) {
    scanf("%c", choice);
    printf("Enter the value to filter: ");
    scanf("%39[^\n]", filterValue); //read string with spaces
}

//processing all CSV files
void processFiles(const char *choice, const char *filterValue, int *totalOrders, double *totalRevenue, double *totalProfit) {
    char filenames[NUMFILES][10] = {"xaa.csv", "xab.csv", "xac.csv", "xad.csv", "xae.csv",
                                     "xaf.csv", "xag.csv", "xah.csv", "xai.csv", "xaj.csv",
                                     "xak.csv", "xal.csv", "xam.csv", "xan.csv", "xao.csv",
                                     "xap.csv", "xaq.csv", "xar.csv", "xas.csv", "xat.csv"};
    for(int i = 0; i < NUMFILES; i++) {
        processFile(filenames[i], choice, filterValue, totalOrders, totalRevenue, totalProfit);
    }
}
```

After setting up my main function, I implemented the supporting functions. In showMenu(), I displayed the filter options. In getUserInput(), I read the user's choice and the filter value, with spaces handling. I implemented the processFiles() function. I stored the names of the twenty CSV files in a simple array and looped through them. For each file, I called processFile() to apply the user's filter and update the right values for orders, revenue, and profit. In this section I didn't use any special algorithms, I just used straightforward sequential processing. The only data structure here is the array that holds the filenames. This part focuses on going through all the files and collecting the required data for user's filter

```

lara@Lara23: ~/os_project
GNU nano 7.2
naive.c *

//processing a single CSV file
void processFile(const char *filename, const char *choice, const char *filterValue, int *totalOrders, double *totalRevenue, double *totalProfit) {
    FILE *fp = fopen(filename, "r");
    if (!fp) {
        printf("Error opening file: %s\n", filename);
        return;
    }

    char line[MAXLINE];
    fgets(line, MAXLINE, fp); // skip header lin
    while(fgets(line, MAXLINE, fp)) {
        SalesRecord record;
        char *token;
        int col = 0;

        //tokenizing line by comma
        token = strtok(line, ",");
        while(token != NULL) {
            switch(col) {
                case 0: strncpy(record.region, token, MAXINPUT); record.region[MAXINPUT-1] = '\0'; break;
                case 1: strncpy(record.country, token, MAXINPUT); record.country[MAXINPUT-1] = '\0'; break;
                case 2: strncpy(record.itemType, token, MAXINPUT); record.itemType[MAXINPUT-1] = '\0'; break;
                case 3: strncpy(record.salesChannel, token, MAXINPUT); record.salesChannel[MAXINPUT-1] = '\0'; break;
                case 8: record.unitsSold = atoi(token); break;
                case 9: record.unitPrice = atof(token); break;
                case 10: record.unitCost = atof(token); break;
            }
            token = strtok(NULL, ",");
            col++;
        }

        // skip invalid lines
        if(col < 11) continue;

        //check filter
        int match = 0;
        switch(*choice) {
            case 'a': if(strcmp(record.region, filterValue) == 0) match = 1; break;
            case 'b': if(strcmp(record.country, filterValue) == 0) match = 1; break;
            case 'c': if(strcmp(record.itemType, filterValue) == 0) match = 1; break;
            case 'd': if(strcmp(record.salesChannel, filterValue) == 0) match = 1; break;
        }

        //updating totals
        if(match) {
            (*totalOrders)++;
            *totalRevenue += record.unitsSold * record.unitPrice;
            *totalProfit += record.unitsSold * (record.unitPrice - record.unitCost);
        }
    }
}

```

In processFile() I open the CSV file and immediately return with an error message if it can't be opened. Then I read and discard the header line so the loop only processes data rows. For each subsequent line I read with fgets(), I store it in a fixed buffer line[MAXLINE]. I create a salesrecord variable on the stack to hold the parsed fields for that row, and I use strtok() to split the line by commas. I track which column I'm at with col, and for each token I copy the textual fields into the corresponding salesrecord char arrays (using strncpy() and forcing a terminating '\0'), and I convert numeric tokens to integers or doubles for unitsSold, unitPrice, and unitCost using atoi() and atof() only when I hit those specific column indices (0..3 for text, 8..10 for numbers). After tokenizing, I do a quick sanity check: if fewer than 11 columns were found (col < 11) I skip the line because it's malformed (invalid because something is missing in it). Then I check the user-chosen filter: depending on choice (a,b,c,d) I compare the chosen field in the record with filterValue using

strcmp(). If they match, I increment the orders counter and add the calculated revenue (unitsSold * unitPrice) and profit (unitsSold * (unitPrice - unitCost)) to the running totals passed by reference

.

2. Multi-processing Approach

The code implementation is as follows:

```
lara@Lara23: ~/os_project
GNU nano 7.2 multiprocessing.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/time.h>
#include <sys/wait.h>
#define NUMFILES 20
#define MAXINPUT 40
#define MAXLINE 200

typedef struct {
    char region[MAXINPUT];
    char country[MAXINPUT];
    char itemType[MAXINPUT];
    char salesChannel[MAXINPUT];
    int unitsSold;
    double unitPrice;
    double unitCost;
} Record;

// list of all filenames
char filenames[NUMFILES][10] = {
    "xaa.csv", "xab.csv", "xac.csv", "xad.csv", "xae.csv",
    "xaf.csv", "xag.csv", "xah.csv", "xai.csv", "xaj.csv",
    "xak.csv", "xal.csv", "xam.csv", "xan.csv", "xao.csv",
    "xap.csv", "xaq.csv", "xar.csv", "xas.csv", "xat.csv"
};
```

It includes standard libraries like stdio.h, stdlib.h, and string.h. And other libraries: unistd.h, sys/time.h, and sys/wait.h are needed for multiprocessing such that unistd.h provides fork, pipe, sys/wait.h to allow the parent to wait for children, and sys/time.h is used to measure time. The constants NUMFILES, MAXINPUT, and MAXLINE for the number of files, maximum string length for record fields (input from user), and maximum line length for reading records, the Record struct stores the fields from each record row, finally the array this simply defines a list of all 20 csv filenames to be processed, storing each name as a string in a fixed-size array so the program can easily loop through them.

```
lara@Lara23: ~/os_project
GNU nano 7.2 multiprocessing.c *
// process a set of files and aggregate totals
void process_files(int startFile, int endFile, char choice, char *value, long long *orders, double *revenue, double *profit) {
    *orders = 0;
    *revenue = 0;
    *profit = 0;

    for(int f=startFile; f<endFile; f++){
        FILE *fp = fopen(filenames[f], "r");
        char line[MAXLINE];
        int first = 1;
        while(fgets(line, MAXLINE, fp)){
            if(first) { first=0; continue; } // skip header

            Record rec={0};
            char buf[MAXLINE];
            strncpy(buf, line, MAXLINE-1);
            buf[MAXLINE-1]='\0';

            char *token = strtok(buf, ",");
            int col=0;
            while(token!=NULL){
                while(*token==' ') token++;
                char *endp = token + strlen(token) -1;
                while(endp>token && (*endp=='\n' || *endp==' ')) *endp--='\0';

                switch(col){
                    case 0: strncpy(rec.region, token, MAXINPUT-1); break;
                    case 1: strncpy(rec.country, token, MAXINPUT-1); break;
                    case 2: strncpy(rec.itemType, token, MAXINPUT-1); break;
                    case 3: strncpy(rec.salesChannel, token, MAXINPUT-1); break;
                    case 8: rec.unitsSold = atoi(token); break;
                    case 9: rec.unitPrice = atof(token); break;
                    case 10: rec.unitCost = atof(token); break;
                }
                token++;
            }
        }
    }
}
```

```

lara@Lara23: ~/os_project
GNU nano 7.2 multiprocessing.c *
    }
    col++;
    token = strtok(NULL, ",");
}

int match=0;
switch(choice){
    case 'a': match = (strcmp(rec.region,value)==0); break;
    case 'b': match = (strcmp(rec.country,value)==0); break;
    case 'c': match = (strcmp(rec.itemType,value)==0); break;
    case 'd': match = (strcmp(rec.salesChannel,value)==0); break;
}
if(match){
    (*orders)++;
    (*revenue) += rec.unitsSold * rec.unitPrice;
    (*profit) += rec.unitsSold * (rec.unitPrice - rec.unitCost);
}
}
fclose(fp);
}
}

```

This function `process_files` handles a subset of CSV files and calculates totals for orders, revenue, and profit. It takes a range of files from `startFile` to `endFile`, a filter choice and value, and pointers to store the running totals. For each file in the range, it opens the file and skips the first line, assuming it's a header. Then, for each data line, it copies the line to a buffer and uses `strtok` to split the fields by commas. It trims spaces and newline characters, and stores the relevant fields in a `Record` struct: string fields for region, country, item type, and sales channel, and numeric fields for units sold, unit price, and unit cost. After parsing, it checks if the record matches the chosen filter and, if it does, updates the totals using the pointers. Finally, it closes the file and moves to the next one, aggregating all results in the given range of files. Later in the next screenshot the main function, it will show these files are divided between different processes so that each process works on its own files.

```

lara@Lara23: ~/os_project
GNU nano 7.2 multiprocessing.c *
int main(){
    char choice;
    char value[MAXINPUT];
    int numProcesses;

    printf("choose a filter option:\n");
    printf("a. region\nb. country\nc. item type\nd. sales channel\n");
    scanf(" %c",&choice);
    printf("enter value: ");
    scanf(" %39[^\n]",value);
    printf("enter number of processes: ");
    scanf("%d",&numProcesses);
    if(numProcesses<0) numProcesses=2;
    if(numProcesses>NUMFILES) numProcesses=NUMFILES; // don't exceed number of files

    struct timeval start,end;
    gettimeofday(&start,NULL);

    int base = NUMFILES / numProcesses;
    int rem = NUMFILES % numProcesses;

    int (*pipefd)[2] = malloc(sizeof(int[2])*numProcesses);
    pid_t *pids = malloc(sizeof(pid_t)*numProcesses);

    for(int p=0;p<numProcesses;p++){
        pipe(pipefd[p]);
        pids[p] = fork();
        if(pids[p]==0) { // child
            close(pipefd[p][0]);

            int startFile = p*base + (p<rem?p:rem);
            int len = base + (p<rem?1:0);
            int endFile = startFile + len;

            long long localOrders=0;
            double localRevenue=0, localProfit=0;

            process_files(startFile,endFile,choice,value,&localOrders,&localRevenue,&localProfit);
        }
    }
}

```

```

        write(pipefd[p][1], &localOrders, sizeof(localOrders));
        write(pipefd[p][1], &localRevenue, sizeof(localRevenue));
        write(pipefd[p][1], &localProfit, sizeof(localProfit));
        close(pipefd[p][1]);
        _exit(0);
    } else {
        close(pipefd[p][1]); // parent closes write
    }
}

long long totalOrders=0;
double totalRevenue=0, totalProfit=0;
for(int p=0;p<numProcesses;p++){
    long long o; double r, pr;
    read(pipefd[p][0], &o, sizeof(o));
    read(pipefd[p][0], &r, sizeof(r));
    read(pipefd[p][0], &pr, sizeof(pr));
    totalOrders+=o;
    totalRevenue+=r;
    totalProfit+=pr;
    close(pipefd[p][0]);
}

for(int p=0;p<numProcesses;p++) waitpid(pids[p], NULL, 0);

gettimeofday(&end, NULL);
double elapsed = (end.tv_sec - start.tv_sec) + (end.tv_usec - start.tv_usec)/1e6;

printf("\nresults for \"%s\":\n", value);
printf("total orders: %lld\n", totalOrders);
printf("total revenue: %.2f\n", totalRevenue);
printf("total profit: %.2f\n", totalProfit);
printf("execution time: %.6f seconds\n", elapsed);

free(pipefd);
free(pids);
return 0;
}

```

Here we have first to explain two main things used: pipes and fork function:

- A pipe is just like a one-way channel between two processes. We make one for each child, and it contains two ends : one for reading and one for writing. parent reads from it, child writes to it. That's how the child sends its results back (orders, revenue, profit).
- Fork is where it works. We call fork and the OS copies the process. Now we got two : the parent and a new child. they both keep running the same code, but fork returns different numbers so they know who's who. Child gets 0, parent gets the child's pid. so the child goes off and does the file processing, writes to the pipe. parent just waits and collects everything later.
- Putting them together and work, we've got multiprocessing. Each process handles its files at the same time, then sends results back through the pipe. Parent grabs it all and combines it. And so works multiprocessing: each process works on its files at the same time another working on its files.

So the main function is the heart of the multiprocessing approach. It starts by getting the user's filter choice, value, and number of processes, and records the start time. It then divides the 20 files bw the requested number of processes using base and rem so each process gets approximately the same number of files. For each process it creates a pipe and forks a child. The child closes the read end of the pipe, processes its assigned files using process_files and writes its local totals back to the parent through the pipe before it exits. The parent closes the write ends and reads the results from all children then sums them to get the overall totals and waits for every child to finish with waitpid, and finally calculates and prints the time along with the totals. This approach allows multiple processes to handle different parts of the dataset at the same time: which is the key of multiprocessing, while the parent waits all children then collects and combines the results.

- The program uses the `gettimeofday()` function in this main function to determine the execution time. After receiving the user's input, it immediately records the time in a struct `timeval` named `start`. The end time is recorded in a different struct `timeval` called `end` once all child processes have completed and the parent has gathered and totaled all results. Subtracting the start time's seconds (`tv_sec`) and microseconds (`tv_usec`) from the end time yields the elapsed time, which is then combined into a single double value in seconds. This provides an accurate estimate of how long the multiprocessing computation took overall, including how long it took for each child to process their files and how long it took the parent to get the results.

3. Multi-threading Approach:

The code implementation is as follows:

```

Select lara@Lara23: ~/os_project
GNU nano 7.2                                multithreading.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <pthread.h>
#include <sys/time.h>
#define NUMFILES 20
#define MAXINPUT 40
#define MAXLINE 200
// struct for a single sales record
typedef struct {
    char region[MAXINPUT];
    char country[MAXINPUT];
    char itemType[MAXINPUT];
    char salesChannel[MAXINPUT];
    int unitsSold;
    double unitPrice;
    double unitCost;
} Record;

// thread data: files assigned to this thread
typedef struct {
    int startFile;           // index of first file
    int endFile;             // index of last+1
    char filterOption;       // 'a'..'d'
    char filterValue[MAXINPUT]; // value to match
    long long totalOrders;   // local totals
    double totalRevenue;
    double totalProfit;
} ThreadData;

// list of filenames
char filenames[NUMFILES][10] = {
    "xaa.csv", "xab.csv", "xac.csv", "xad.csv", "xae.csv",
    "xaf.csv", "xag.csv", "xah.csv", "xai.csv", "xaj.csv",
    "xak.csv", "xal.csv", "xam.csv", "xan.csv", "xao.csv",
    "xap.csv", "xaq.csv", "xar.csv", "xas.csv", "xat.csv"
};

```

Mostly everything already explained except for : `pthread.h` library, and `threadData` struct:

- **pthread.h library** which lets us create multiple threads, manage them, and synchronize their work .
- **ThreadData struct** holds a pointer to the big array of all records, the start and end indices that this thread gonna handle, the filter option and value and local totals for orders revenue and profit. This allows each thread to know which part of the data it should process and to store its results independently.

```

// function to process files assigned to thread
void process_files(ThreadData *td) {
    td->totalOrders = 0;
    td->totalRevenue = 0.0;
    td->totalProfit = 0.0;

    char line[MAXLINE];
    for (int f = td->startFile; f < td->endFile; f++) {
        FILE *fp = fopen(filenamees[f], "r");
        if (!fp) continue;

        int first = 1;
        while (fgets(line, MAXLINE, fp)) {
            if (first) { first = 0; continue; } // skip header

            Record r;
            memset(&r, 0, sizeof(r));
            char buf[MAXLINE];
            strncpy(buf, line, MAXLINE-1); buf[MAXLINE-1]='\0';

            char *token = strtok(buf, ",");
            int col = 0;
            while (token != NULL) {
                while(*token == ' ') token++;
                char *endp = token + strlen(token) -1;
                while(endp>token && (*endp=='\n' || *endp=='\r' || *endp==' ')) *endp--='\0';

                switch(col){
                    case 0: strncpy(r.region, token, MAXINPUT-1); break;
                    case 1: strncpy(r.country, token, MAXINPUT-1); break;
                    case 2: strncpy(r.itemType, token, MAXINPUT-1); break;
                    case 3: strncpy(r.salesChannel, token, MAXINPUT-1); break;
                    case 8: r.unitsSold = atoi(token); break;
                    case 9: r.unitPrice = atof(token); break;
                    case 10: r.unitCost = atof(token); break;
                }

                col++;
                token = strtok(NULL, ",");
            }

            int match = 0;
            switch(td->filterOption){
                case 'a': match = strcmp(r.region, td->filterValue)==0; break;
                case 'b': match = strcmp(r.country, td->filterValue)==0; break;
                case 'c': match = strcmp(r.itemType, td->filterValue)==0; break;
                case 'd': match = strcmp(r.salesChannel, td->filterValue)==0; break;
            }

            if (match) {
                td->totalOrders++;
                td->totalRevenue += r.unitsSold * r.unitPrice;
                td->totalProfit += r.unitsSold * (r.unitPrice - r.unitCost);
            }
            fclose(fp);
        }
    }
}

```

This function handles all the files assigned to a thread. It starts by initializing the thread's local totals for orders, revenue, and profit. Then, for each file in the assigned range, it opens the file and reads it line by line, skipping the header. Each line is split into fields using strtok, and spaces or newline characters are trimmed. The relevant fields are stored in a Record struct, and the thread checks if the record matches the chosen filter. If it does, the thread updates its local totals. After processing all lines in a file, it closes the file and moves on to the next one. This function allows each thread to independently process a subset of files and accumulate results without interfering with other threads.

```
// thread function
void *thread_func(void *arg) {
    ThreadData *td = (ThreadData*)arg;
    process_files(td);
    return NULL;
}
```

This is the function that each thread runs when it starts. It receives a pointer to a ThreadData struct, which tells the thread which files or records to process and where to store its results. Inside, it simply calls process_files with that data. After finishing, the thread returns NULL. This function acts as a small wrapper so pthread_create can start the thread with the correct work.

```
int main() {
    char choice;
    char value[MAXINPUT];
    int numThreads;

    printf("choose a filter option:\n");
    printf("a. region\nb. country\nc. item type\nd. sales channel\n");
    if(scanf(" %c",&choice)!=1) return 1;
    printf("enter value: ");
    scanf(" %39[^\n]", value);
    printf("enter number of threads: ");
    if(scanf("%d",&numThreads)!=1) return 1;

    if(numThreads <=0) numThreads=2;
    if(numThreads > NUMFILES) numThreads=NUMFILES;

    pthread_t *threads = malloc(sizeof(pthread_t)*numThreads);
    ThreadData *tdata = malloc(sizeof(ThreadData)*numThreads);

    // divide files among threads
    int base = NUMFILES / numThreads;
    int rem = NUMFILES % numThreads;
    int start = 0;
    for(int t=0;t<numThreads;t++){
        int len = base + (t<rem?1:0);
        tdata[t].startFile = start;
        tdata[t].endFile = start + len;
        tdata[t].filterOption = choice;
        strncpy(tdata[t].filterValue, value, MAXINPUT-1);
        tdata[t].filterValue[MAXINPUT-1]='\0';
        start += len;
    }

    struct timeval startTime, endTime;
    gettimeofday(&startTime,NULL);

    // start threads
    for(int t=0;t<numThreads;t++){
        pthread_create(&threads[t],NULL,thread_func,&tdata[t]);
    }

    long long totalOrders=0;
    double totalRevenue=0.0, totalProfit=0.0;

    // join threads and aggregate
    for(int t=0;t<numThreads;t++){
        pthread_join(threads[t],NULL);
        totalOrders += tdata[t].totalOrders;
        totalRevenue += tdata[t].totalRevenue;
        totalProfit += tdata[t].totalProfit;
    }

    gettimeofday(&endTime,NULL);
    double elapsed = (endTime.tv_sec - startTime.tv_sec) + (endTime.tv_usec - startTime.tv_usec)/1e6;

    printf("\nresults for \"%s\":\n", value);
    printf("total orders: %lld\n", totalOrders);
    printf("total revenue: %.2lf\n", totalRevenue);
    printf("total profit: %.2lf\n", totalProfit);
    printf("execution time: %.6f seconds\n", elapsed);

    free(threads);
    free(tdata);
    return 0;
}
```

This main function is the heart of the multithreading approach. It starts by asking the user for a filter option, the value to filter by, and the number of threads. Then it allocates memory for the thread IDs and ThreadData structs. The code divides the 20 CSV files among the threads so each one gets roughly the same number of files, handling any remainder with the rem logic. Each ThreadData struct is initialized with the start and end file indices, the filter option, and the filter value.

Next, it records the start time with gettimeofday and creates the threads using pthread_create, passing each thread its ThreadData. After all threads start, the main function waits for them to finish using pthread_join and sums the totals from each thread to get the overall orders, revenue, and profit.

Finally, it records the end time, calculates the elapsed time in seconds, prints the results, and frees the allocated memory. This setup allows multiple threads to process different parts of the dataset simultaneously, improving performance by parallelizing the file processing.

Results

1. Naïve Approach

In the Naive approach, e.g. there is no child processes or threads, the execution time for the final result is e.g. **0.975 seconds** for Asia filter as shown in figure 7, takes long time

```
lara@Lara23:/mnt/c/Users/hp$ cd ~/os_project
```

```
lara@Lara23:~/os_project$ ./naive
```

```
Choose a filter option:
```

- a. Region
- b. Country
- c. Item Type
- d. Sales Channel

```
Enter choice (a/b/c/d): a
```

```
Enter the value to filter: Asia
```

```
Results for "Asia":
```

```
Total Orders: 146171
```

```
Total Revenue: 194511792496.21
```

```
Total Profit: 57370565323.36
```

```
Execution Time: 0.975 seconds
```

```
lara@Lara23:~/os_project$ ./naive
```

```
Choose a filter option:
```

- a. Region
- b. Country
- c. Item Type
- d. Sales Channel

```
Enter choice (a/b/c/d): b
```

```
Enter the value to filter: Palau
```

```
Results for "Palau":
```

```
Total Orders: 5327
```

```
Total Revenue: 7123254338.20
```

```
Total Profit: 2099038669.99
```

```
Execution Time: 0.950 seconds
```

Figure 7: Naive Results

2. Multi-threading approach

2 Threads :

```
lara@Lara23:~/os_project$ ./multithreading
Choose a filter option:
a. Region
b. Country
c. Item Type
d. Sales Channel
Enter choice (a/b/c/d): a
Enter the value to filter: Asia
Enter number of threads (2,4,8,16,20): 2

Results for "Asia":
Total Orders: 146171
Total Revenue: 194511792496.21
Total Profit: 57370565323.36
Execution Time: 0.022556 seconds
```

Figure 8: 2 Threads

4 Threads case:

```
lara@Lara23:~/os_project$ ./multithreading
Choose a filter option:
a. Region
b. Country
c. Item Type
d. Sales Channel
Enter choice (a/b/c/d): a
Enter the value to filter: Asia
Enter number of threads (2,4,8,16,20): 4

Results for "Asia":
Total Orders: 146171
Total Revenue: 194511792496.21
Total Profit: 57370565323.36
Execution Time: 0.039162 seconds
```

Figure 9: 4 Threads

8 Threads:

```
lara@Lara23:~/os_project$ ./multithreading
Choose a filter option:
a. Region
b. Country
c. Item Type
d. Sales Channel
Enter choice (a/b/c/d): a
Enter the value to filter: Asia
Enter number of threads (2,4,8,16,20): 8

Results for "Asia":
Total Orders: 146171
Total Revenue: 194511792496.21
Total Profit: 57370565323.36
Execution Time: 0.017425 seconds
```

Figure 10: 8 Threads

16 Threads:

```
lara@Lara23:~/os_project$ ./multithreading
Choose a filter option:
a. Region
b. Country
c. Item Type
d. Sales Channel
Enter choice (a/b/c/d): a
Enter the value to filter: Asia
Enter number of threads (2,4,8,16,20): 16

Results for "Asia":
Total Orders: 146171
Total Revenue: 194511792496.21
Total Profit: 57370565323.36
Execution Time: 0.017308 seconds
```

Figure 11: 16 Threads

20 Threads

```
lara@Lara23:~/os_project$ ./multithreading
Choose a filter option:
a. Region
b. Country
c. Item Type
d. Sales Channel
Enter choice (a/b/c/d): a
Enter the value to filter: Asia
Enter number of threads (2,4,8,16,20): 20

Results for "Asia":
Total Orders: 146171
Total Revenue: 194511792496.21
Total Profit: 57370565323.36
Execution Time: 0.018072 seconds
```

Figure 12: 20 Threads

Since we always got the same results but different time so its working well .here the multithreading results show that adding more threads doesn't always make the program faster. With 2 threads, the execution time was 0.022 seconds, but using 4 threads increased slightly to 0.039 seconds, likely due to how the files were divided and memory access patterns. Using 8 and 16 threads achieved the best performance around 0.017 to 0.018 : **0.017308 seconds** because the workload was split efficiently across the CPU cores. And exactly at 16 we used all the environment threads so we got the best performance. When 20 threads were used, the time increased slightly to 0.018 seconds because some threads processed very small chunks, adding overhead in thread management and memory access. This shows that multithreading helps when the workload is large and divided properly, but too many threads with uneven chunks slightly reduce efficiency. So because my CPU environment provides 12 hardware threads, creating more than 12 threads adds overhead without real parallel speedup. Threads beyond the available cores compete for CPU time, which can slightly increase execution time. This explains why 8 to 16 threads give the best performance and so efficiency decreases and time increases.

3. Multi-processing Approach

2 Processes:

```
Lara@Lara23:~/os_project$ ./multiprocessing
Choose a filter option:
a. Region
b. Country
c. Item Type
d. Sales Channel
Enter choice (a/b/c/d): a
Enter the value to filter: Asia
Enter number of processes (2,4,8,16,20): 2

Results for "Asia":
Total Orders: 146171
Total Revenue: 194511792496.21
Total Profit: 57370565323.36
Execution Time: 0.844644 seconds
```

Figure 13: 2 Processes

4 Processes:

```
lara@Lara23:~/os_project$ ./multiprocessing
Choose a filter option:
a. Region
b. Country
c. Item Type
d. Sales Channel
Enter choice (a/b/c/d): a
Enter the value to filter: Asia
Enter number of processes (2,4,8,16,20): 4

Results for "Asia":
Total Orders: 146171
Total Revenue: 194511792496.21
Total Profit: 57370565323.36
Execution Time: 0.824950 seconds
```

Figure 14: 4 Processes

8 Processes:

```
lara@Lara23:~/os_project$ ./multiprocessing
Choose a filter option:
a. Region
b. Country
c. Item Type
d. Sales Channel
Enter choice (a/b/c/d): a
Enter the value to filter: Asia
Enter number of processes (2,4,8,16,20): 8

Results for "Asia":
Total Orders: 146171
Total Revenue: 194511792496.21
Total Profit: 57370565323.36
Execution Time: 0.809652 seconds
```

Figure 15: 8 Processes

16 Processes:

```
lara@Lara23:~/os_project$ ./multiprocessing
Choose a filter option:
a. Region
b. Country
c. Item Type
d. Sales Channel
Enter choice (a/b/c/d): a
Enter the value to filter: Asia
Enter number of processes (2,4,8,16,20): 16

Results for "Asia":
Total Orders: 146171
Total Revenue: 194511792496.21
Total Profit: 57370565323.36
Execution Time: 0.846263 seconds
```

Figure 16: 16 Processes

20 Processes:

```
lara@Lara23:~/os_project$ ./multiprocessing
Choose a filter option:
a. Region
b. Country
c. Item Type
d. Sales Channel
Enter choice (a/b/c/d): a
Enter the value to filter: Asia
Enter number of processes (2,4,8,16,20): 20

Results for "Asia":
Total Orders: 146171
Total Revenue: 194511792496.21
Total Profit: 57370565323.36
Execution Time: 0.860640 seconds
```

Figure 17: 20 Processes

Also here since we got the same results in the 5 cases but different time so the program works correctly, The results show that as we increase the number of processes from 2 to 8, the execution time gradually decreases, reaching the best performance at 8 processes with **0.809652 seconds**. This makes sense because your CPU has 6 cores. Using 8 processes allows nearly full utilization of all cores without excessive overhead. When we go beyond 8 processes (16 or 20), the execution time starts to increase slightly. This happens because the extra processes exceed the number of physical cores, causing **context switching and CPU scheduling overhead**, which slows down execution instead of speeding it up.

So in summary, **8 processes give the highest performance** because they efficiently use all available cores without introducing significant overhead. Using fewer processes underutilizes the CPU, and using more than the number of cores adds overhead that negates the benefits of parallelism.

*** Note

After several executions of the naive approach, its execution time dropped to **0.422 seconds** as shown in figure 18, making it faster than the multiprocessing approach. This improvement is likely due to **caching effects**. The first time the naive approach runs, the system needs to read all the CSV files from disk and load them into memory. After that, the data is already cached in the operating system's memory and possibly in the CPU caches, so subsequent runs do not need to perform as many slow disk reads. This reduces the overall execution time.

In short, the naive approach benefits from data being cached in memory, which allows it to process the dataset faster than the multiprocessing approach, even though it still does not surpass the multithreading approach which was already optimized for concurrent memory access.

```
lara@Lara23:~/os_project$ ./naive
Choose a filter option:
a. Region
b. Country
c. Item Type
d. Sales Channel
Enter choice (a/b/c/d): a
Enter the value to filter: Asia

Results for "Asia":
Total Orders: 146171
Total Revenue: 194511792496.21
Total Profit: 57370565323.36
Execution Time: 0.422 seconds
```

Figure 18: Naive last attempt

Results Discussion:

The results of the three approaches—naive, multithreading, and multiprocessing—show interesting patterns in performance and behavior. The **naive approach** initially had a longer execution time of around 0.975 seconds because it had to read and process all CSV files sequentially in a single thread. After repeated runs, its execution time dropped to 0.422 seconds. This is likely due to caching effects. After the first execution, parts of the dataset and related file information were already in memory, so subsequent runs required less disk access, making the naive approach faster than the initial runs and even faster than the multiprocessing approach in later attempts. The **multithreading approach** consistently delivered the best performance with execution times between 0.013 and 0.018 seconds depending on the number of threads. Threads share the same memory space which avoids expensive memory duplication. They efficiently divide the work across CPU cores while keeping overhead low. Performance improved as the number of threads increased up to the hardware limit of 12 threads. Beyond that, adding more threads did not improve execution time and sometimes introduced slight overhead because the system had to manage more threads than available physical or logical cores. Execution time for 4 threads was slightly higher than 2 in some cases due to small variations in scheduling but the overall trend showed improvement up to the optimal number of threads. The **multiprocessing approach** performed worse than multithreading with execution times between 0.809 and 0.860 seconds. Creating separate processes duplicates memory for each process and requires inter-process communication to combine results which adds overhead. Its best performance was observed with 8 processes, which closely matches the number of physical cores. Adding more processes beyond that caused context switching and increased overhead, slowing execution slightly. Overall, multithreading benefits the most from shared memory and low overhead. The naive approach becomes very fast after caching occurs. Multiprocessing is limited by memory duplication and communication overhead, making it less efficient than multithreading for this dataset and environment.

Performance Measuring

Table 1: Numerical Results

Case	Attribute	Instance	Total Orders	Total Revenue	Total Profit
I	Region	e.g. Europe	259411	34592250783.34	101930686144.31
II	Country	e.g. Ireland	5409	7281352797.70	2166269387.74
III	Item Type	e.g. Clothes	83123	45432507052.00	30532241196.00
IV	Sales Channel	e.g. Online	500140	664552299300.13	196009323723.93

In measuring the performance I took the filter value **Region-Asia** (the one that I provided run attempts photos for – for each case) and I assumed the first Naïve run with 0.975 sec.

Table 2: Performance Table

Approach		Time	Performance
Naïve		0.975 sec	1.0256410
Multiprocessing	2 child Processes	0.844644 sec	1.1839307
	4 child Processes	0.824950 sec	1.2121946
	8 child Processes	0.809652 sec	1.2350985
	16 child Processes	0.846263 sec	1.1816657
	20 child Processes	0.860640 sec	1.1619233
Multithreading	2 Threads	0.022556 sec	44.334102
	4 Threads	0.039162 sec	25.534957
	8 Threads	0.017425 sec	57.388809
	16 Threads	0.017308 sec	57.776751
	20 Threads	0.018072 sec	55.334123

In our program, which processes a large dataset of sales records, we used three approaches: Naive, Multiprocessing, and Multithreading. We measured the execution time for each approach and put the results in the table above. The slowest approach is Naive because it runs sequentially on a single thread. Its execution time in the first run was 0.975 seconds.

Both Multiprocessing and Multithreading improve performance by dividing the work. In Multiprocessing, using two processes took 0.844 seconds. Increasing to four processes gave a small improvement. The best performance was with eight processes at 0.809 seconds, which makes sense because my working environment has 6 physical cores. Using more than 8 processes, like 16 or 20, made the execution slightly slower because managing extra processes adds overhead.

Multithreading gave the best performance by far. 2 threads already ran the program very fast in 0.022 seconds. The fastest was 16 threads at 0.0173 seconds. Threads share memory, so there is less overhead than processes, and the mutex we used adds only a small delay while making sure totals are updated in a safe way. Using more threads than the CPU supports(12) like 20, caused a small increase in time, but overall it stayed actually fast.

Overall, splitting the work between multiple threads or processes clearly speeds up the program. Multithreading is the fastest approach in this case, while Naive is the slowest, and Multiprocessing sits in the middle.

***Speedups can be simply calculated by dividing the performance of naïve to the selected approach's performance

Amdahl's Law Calculations (Theoretical Speedup)

Amdahl's Law states that the maximum speedup S_p of a parallel program is limited by the serial portion of the code. It is expressed as:

$$S_p = \frac{1}{S + \frac{1-S}{P}}$$

Where:

SSS is the fraction of the program that is serial and cannot be parallelized.

PPP is the number of cores, threads, or processes used.

- **Estimation of Serial Portion**

From the code implemented in our project:

- The **Naive approach** is fully serial.
- For **multithreading**, most operations such as reading files, filtering records, and performing calculations are parallelizable.
- Serial tasks include program initialization, user input, final aggregation of results, and printing outputs.

We can estimate the serial portion using the measured execution times.

Measured execution times:

- Naive approach: 0.975 seconds
- Multithreading with 16 threads: 0.017308 seconds

Observed speedup using 16 threads:

$$\text{Speedup}_{16} = \frac{T_{\text{Naive}}}{T_{16 \text{ threads}}} = \frac{0.975}{0.017308} \approx 56.4$$

From this, the serial portion can be roughly estimated:

$$S \approx \frac{1}{\text{Speedup}_{16}} \approx \frac{1}{56.4} \approx 0.0177 \quad (\text{or } 1.77\%)$$

So the serial part of my code is very small approximately 1–2%.

- **Maximum Theoretical Speedup**

Considering our system provides 12 physical threads/cores, the maximum theoretical speedup according to Amdahl's Law is:

$$S_{\max} = \frac{1}{S + \frac{1-S}{P}} = \frac{1}{0.0177 + \frac{0.9823}{12}} = \frac{1}{0.0177 + 0.0819} = \frac{1}{0.0996} \approx 10.04$$

Therefore, the maximum achievable speedup with 12 cores is approximately 10. Even though our observed speedup with 16 threads exceeds this value, this is due to **caching and memory optimizations** in the system.

- **Optimal Number of Threads and Processes**

- **Multithreading:** Performance improves up to 8–16 threads. But adding more threads does not significantly improve performance because our CPU has 12 hardware threads.
- **Multiprocessing:** The best performance was observed with 8 child processes. Our CPU has 6 physical cores, and increasing the number of processes after this creates additional overhead, context switching, and inter-process communication delays.

- **Comparison: Actual vs. Theoretical Speedup**

Table 3: Actual vs. Theoretical Speedup

Approach	Observed Speedup	Theoretical Max Speedup
Multithreading 16 threads	56.4	10
Multiprocessing 8 processes	1.235	6

Summary :

- Multithreading exhibits extremely high speedup due to shared memory and caching effects, exceeding the theoretical limit predicted by Amdahl's Law.
- Multiprocessing speedup is closer to theoretical predictions because each process maintains its own memory space, introducing more overhead, not too close but relatively.
- Using more threads or processes than available hardware resources leads to performance degradation due to context switching and memory overhead. The Amdahl's Law analysis confirms the following:
 - The serial portion of our code is very small, approximately 1.77%.
 - The maximum speedup is limited by the serial portion and the number of available cores.
 - Multithreading outperforms multiprocessing due to low overhead and shared memory.

The optimal number of threads should match the number of hardware threads (12), while the optimal number of processes should match the number of physical cores (6).

- Performance can degrade beyond these numbers due to context switching, memory overhead, and inter-process communication delays.

Conclusion

In this project, I implemented three approaches : Naive, Multiprocessing, and Multithreading to filter and analyze a 1 million sales records dataset. The goal was to compare their performance and understand the benefits of parallel computing (or the cons). The Naive approach executed sequentially and was initially the slowest, though repeated runs became faster due to caching. Multiprocessing reduced execution time by distributing the workload across multiple processes, with 8 processes performing best due to the 6 available CPU cores. While Multithreading provided the highest performance, with 16 threads achieving the lowest execution time. Threads are lightweight and share memory, which minimizes overhead compared to processes. It turned out that using more threads than the hardware supports caused slowdowns, and that obviously shows the importance of matching parallelism to system resources.

Overall, this project demonstrates that parallel computing greatly improves efficiency, with highlights to the impact of choosing the right number of threads or child processes, and shows that multithreading is the optimal approach for this dataset on my system.

References

- [1] "Difference between Multiprocessing and Multithreading," [Online]. Available:
https://www.geeksforgeeks.org/operating-systems/difference-between-multiprocessing-and-multithreading/?utm_source=. [Accessed 22 November 2025].
- [2] "POSIX Threads (pthreads) Tutorial," [Online]. Available:
https://www.yolinux.com/TUTORIALS/LinuxTutorialPosixThreads.html?utm_source=.
[Accessed 22 November 2025].
- [3] "Linux Process vs Thread," [Online]. Available: https://www.baeldung.com/linux/process-vs-thread?utm_source=. [Accessed 22 November 2025].